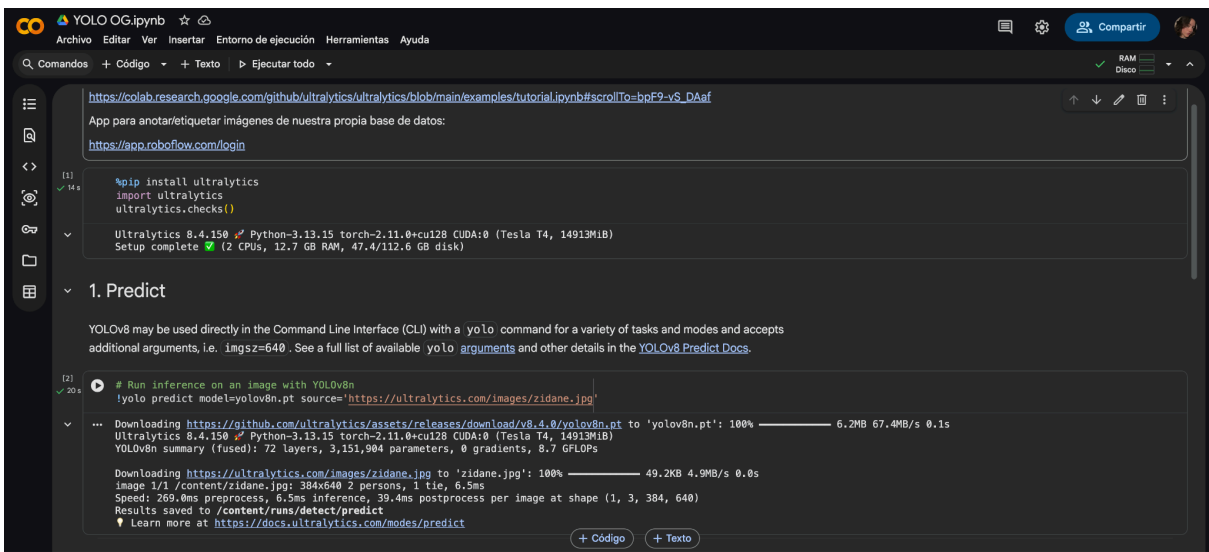


Ejercicio 1 - Visión computacional

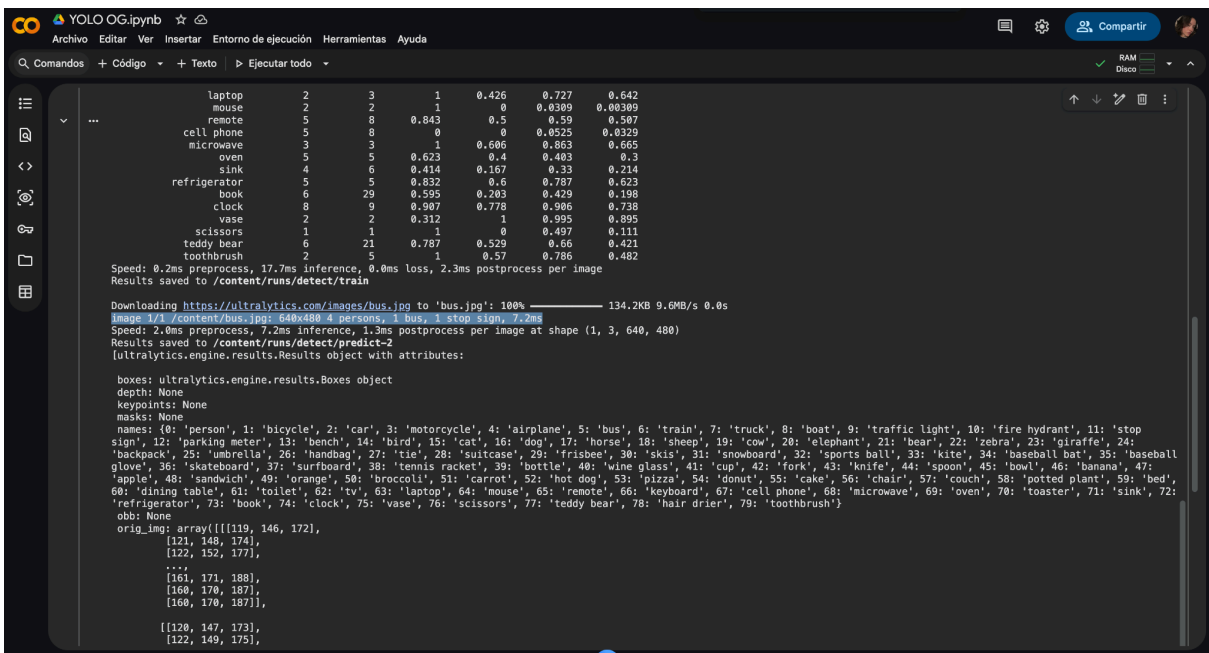
Notebook : 13 YOLO ultralytics.ipynb

Capturas

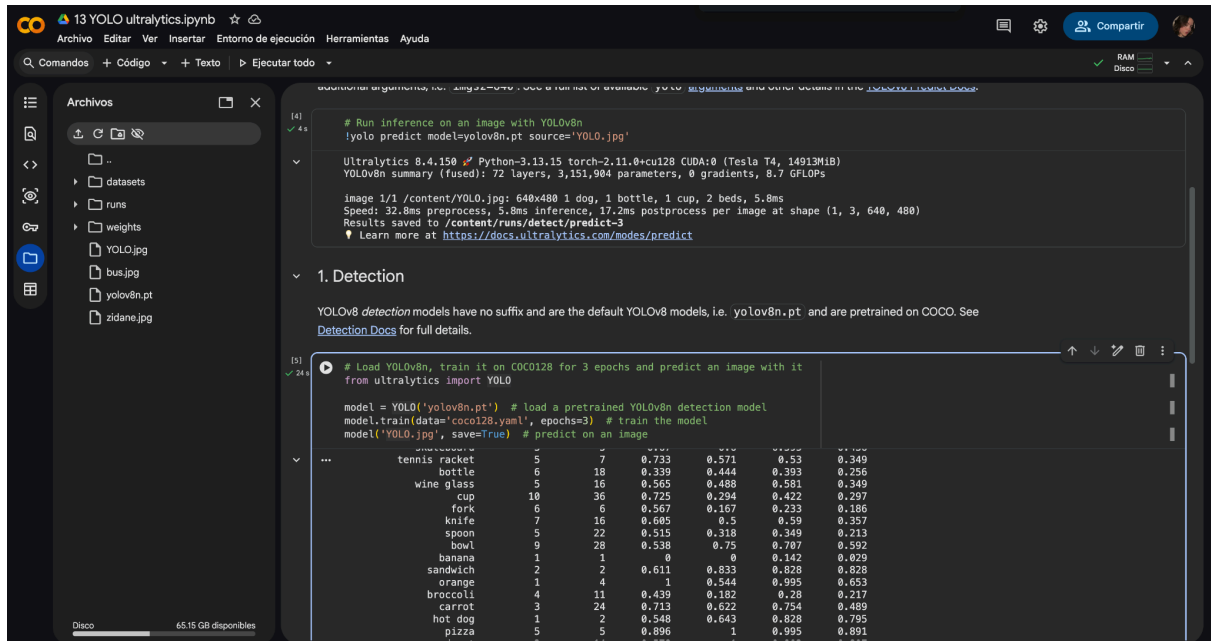
salida de predicción — zidane.jpg



salida de predicción — bus.jpg



Mi foto, celda CLI



The screenshot shows a JupyterLab environment with a file explorer on the left containing folders like 'datasets', 'runs', 'weights', and files like 'YOLO.jpg', 'bus.jpg', 'yolov8n.pt', and 'zidane.jpg'. The main area displays a CLI cell with the following content:

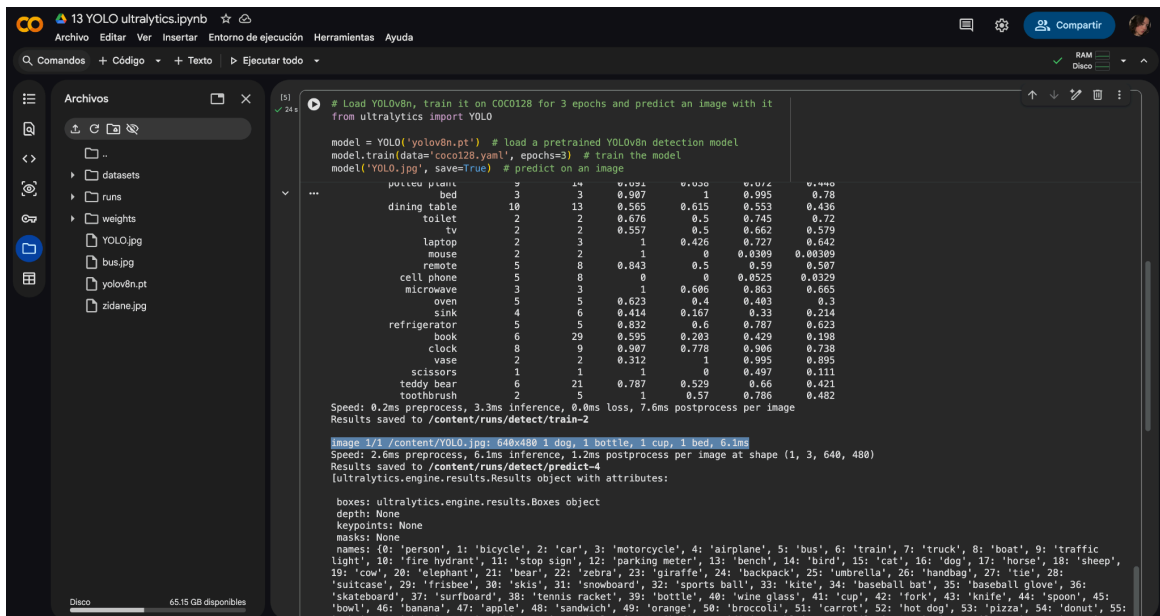
```
[4]: # Run inference on an image with YOLOv8n
yolo predict model=yolov8n.pt source='YOLO.jpg'
```

Below the code, the environment details are shown: Ultralytics 8.4.150, Python-3.13.15, torch-2.11.0+cu128, CUDA:0 (Tesla T4, 14913MiB). The YOLOv8n summary indicates 72 layers, 3,151,984 parameters, 0 gradients, and 8.7 GFLOPs. The inference results for the image 'YOLO.jpg' (640x480) are: 1 dog, 1 bottle, 1 cup, 2 beds, 5.8ms preprocess, 32.8ms inference, 17.2ms postprocess per image at shape (1, 3, 640, 480). Results are saved to /content/runs/detect/predict-3.

The cell then shows the output of the '1. Detection' command, which lists the detected objects and their confidence scores:

Class	Confidence
tennis racket	0.733
bottle	0.571
wine glass	0.565
cup	0.725
fork	0.567
knife	0.605
spoon	0.515
bowl	0.538
banana	0.142
sandwich	0.611
orange	0.544
broccoli	0.439
carrot	0.713
hot dog	0.548
pizza	0.896

Mi foto, celda Python model



The screenshot shows a JupyterLab environment with a file explorer on the left containing folders like 'datasets', 'runs', 'weights', and files like 'YOLO.jpg', 'bus.jpg', 'yolov8n.pt', and 'zidane.jpg'. The main area displays a Python cell with the following content:

```
[5]: # Load YOLOv8n, train it on COCO128 for 3 epochs and predict an image with it
from ultralytics import YOLO

model = YOLO('yolov8n.pt') # load a pretrained YOLOv8n detection model
model.train(data='coco128.yaml', epochs=3) # train the model
model('YOLO.jpg', save=True) # predict on an image
```

Below the code, the environment details are shown: Ultralytics 8.4.150, Python-3.13.15, torch-2.11.0+cu128, CUDA:0 (Tesla T4, 14913MiB). The YOLOv8n summary indicates 72 layers, 3,151,984 parameters, 0 gradients, and 8.7 GFLOPs. The inference results for the image 'YOLO.jpg' (640x480) are: 1 dog, 1 bottle, 1 cup, 1 bed, 6.1ms preprocess, 2.6ms inference, 1.2ms postprocess per image at shape (1, 3, 640, 480). Results are saved to /content/runs/detect/predict-4.

The cell then shows the output of the 'ultralytics.engine.results.Results object with attributes:' command, which lists the detected objects and their confidence scores:

Class	Confidence
bed	0.987
dining table	0.665
toilet	0.676
tv	0.557
laptop	0.426
mouse	0.0389
remote	0.5
cell phone	0.0525
microwave	0.606
oven	0.623
sink	0.414
refrigerator	0.832
book	0.595
clock	0.907
vase	0.312
scissors	0.1
teddy bear	0.787
toothbrush	0.57

Análisis

¿Qué clases detectó YOLO en las fotos de Ultralytics y cuáles en la mía?

En zidane.jpg: person (x2) y tie. En bus.jpg: person (x4), bus y stop sign. En mi foto: dog, bottle, cup y bed un conjunto de clases totalmente distinto porque pues es completamente distinta.

¿Algún objeto evidente de mi foto no salió etiquetado? ¿Por qué podría pasar?

Sí, dos casos claros. La laptop, visible y abierta sobre la cama, no se detectó, a pesar de que laptop sí es una clase de COCO lo más probable es que por el reflejo de la laptop mas los stickers de la funda rompen su silueta característica. La mochila negra tampoco salió, probablemente porque no se ve completa. El juguete de dinosaurio en el buró tampoco se detectó, pero este caso es distinto: COCO no tiene una clase de “juguete” ni “dinosaurio”, así que no hay forma de que el modelo lo etiquete sin importar qué tan visible esté.

¿La predicción de la celda CLI y la de model(...) coinciden sobre mi misma imagen?

Casi: coinciden en dog, bottle y cup, pero el CLI reportó 2 beds y la celda de Python solo 1 bed. La razón no es una diferencia entre la interfaz CLI y la API de Python en sí, sino que cada celda volvió a entrenar el modelo desde cero antes de predecir (el log muestra “train” en la primera corrida y “train-2” en la segunda) — como el entrenamiento tiene aleatoriedad (aumentos de datos, orden de batches), terminé comparando dos modelos afinados ligeramente distintos, no la misma red vista desde dos formas de invocarla. Esa es la causa más probable del doble conteo de cama en una corrida y no en la otra.

Reto

Cambiar solo el modelo de la predicción CLI a *yolov8s.pt*

```
[6]
✓ 10s # Run inference on an image with YOLOv8n
!yolo predict model=yolov8s.pt source='YOL0.jpg'

Ultralytics 8.4.150 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLOv8s summary (fused): 72 layers, 11,156,544 parameters, 0 gradients, 28.6 GFLOPs

image 1/1 /content/YOL0.jpg: 640x480 1 dog, 1 bottle, 1 bed, 1 laptop, 12.2ms
Speed: 86.7ms preprocess, 12.2ms inference, 33.8ms postprocess per image at shape (1, 3, 640, 480)
Results saved to /content/runs/detect/predict
🔗 Learn more at https://docs.ultralytics.com/modes/predict
```

Con el modelo más grande sí apareció laptop, la clase estaba bien etiquetada en COCO, el problema no era eso, sino que *yolov8n* (el modelo más chico y rápido) no tenía suficiente capacidad para distinguirla en esas condiciones de poco contraste. Además, el falso positivo de cup que había dado *yolov8n* ya no esta con **yolov8s** lo cual es otra señal de que el modelo más grande generaliza mejor sobre esta imagen, a costa de ser más lento (28.6 GFLOPs vs. los 8.9 de *yolov8n*).